Programming for AI	Lab 03 Introduction to Conditional Statements, Loops, Functions, Built-in Functions and Lambda Expressions using Python

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES

Fall 2026

Aims and Objectives

1. To understand the basic concepts of control flow in Python.
2. To implement conditional statements using if, if-else, and if-elif-else.
3. To use match-case for handling multiple choices.
4. To understand and implement for and while loops.
5. To practice loop control statements such as break, continue, and pass.
6. To create reusable and modular code using user-defined functions.
7. To understand function arguments, parameters, return values, and variable scope.
8. To use useful built-in functions such as enumerate() and zip().
9. To understand and implement anonymous functions using lambda.

Introduction

In this lab we will be covering the following topics

Python provides several control-flow and programming constructs that allow developers to make decisions, repeat tasks, and organize code efficiently. In this lab, students will learn conditional statements such as if-else and match-case, different forms of loops, and loop control statements. The lab will also introduce user-defined functions for creating reusable and modular code, along with parameters, arguments, return values, and variable scope. Finally, students will practice useful built-in functions such as enumerate() and zip(), as well as anonymous functions using lambda, to write concise and efficient Python programs.

Conditional Statements

The if statement is used to execute a block of code only when a specified condition evaluates to True. It is one of the basic decision-making statements in Python.
Python uses indentation to define the body of an if statement.
The if-else statement is used when a program needs to choose between two alternatives. If the condition is true, the if block executes; otherwise, the else block executes.

```
age = 20
has_id = True

if age >= 18:
    if has_id:
        print("Entry allowed")
    else:
        print("ID is required")
else:
    print("Entry not allowed")
```

Match-Case Statement

The **match-case statement** is used for pattern matching and handling multiple possible values. It was introduced in Python 3.10 and can be used for situations similar to a traditional switch statement in other programming languages.

The underscore _ is commonly used as the default case.

```
choice = 2

match choice:
    case 1:
        print("Add Student")
    case 2:
        print("Display Students")
    case 3:
        print("Exit")
    case _:
        print("Invalid choice")
```

FOR LOOP

A **for loop** is used to iterate over the elements of a sequence or iterable such as a list, tuple, string, dictionary, or range of numbers.

```
for i in range(5):
    print(i)
```

The range() function generates a sequence of numbers and is frequently used with for loops.
range(stop)
range(start, stop)
range(start, stop, step)

```
for i in range(1, 6):
    print(i)
for i in range(0, 11, 2):
    print(i)
```

A negative step can be used to iterate through a sequence in reverse order.

```
for i in range(10, 0, -1):
    print(i)
```

A for loop can directly access each element of a list.

```
students = ["Ali", "Sara", "Ahmed"]

for student in students:
    print(student)
```

A string is an iterable object, so a for loop can be used to process each character individually.

```
word = "Python"

for character in word:
    print(character)
```

A dictionary can be iterated through its keys, values, or key-value pairs.

```
student = {
```

```
    "name": "Ali",
    "age": 20,
    "marks": 85
}

for key, value in student.items():
    print(key, value)
```

Python allows an else block to be associated with a for loop. The else block executes when the loop completes normally without encountering a break statement.

```
for i in range(5):
    print(i)
else:
    print("Loop completed")
```

The **pass statement** is a null statement. It does not perform any operation and is generally used as a placeholder when a statement is syntactically required.

```
for i in range(5):
    if i == 2:
        pass
    else:
        print(i)
```

A while loop can also have an else block. The else block executes when the loop condition becomes false and the loop ends normally.

```
count = 1

while count <= 3:
    print(count)
    count += 1
else:
    print("Loop completed")
```

USER-DEFINED FUNCTIONS

A **function** is a reusable block of code designed to perform a specific task. Functions help divide a large program into smaller, manageable, and reusable components. Python provides many built-in functions, but programmers can also create their own functions according to the requirements of a program. These are called **user-defined functions**.

A function is created using the def keyword. Once defined, it can be called multiple times from different parts of the program.

```
def greet():
    print("Welcome to Python Programming")

greet()
```

```
def greet(name):
    print("Hello", name)

greet("Ali")
greet("Sara")
```

In **positional arguments**, values are assigned to parameters according to their position in the function call.

In **keyword arguments**, the parameter name is explicitly specified when calling the function. This allows arguments to be passed in a different order.

```
def student_info(name, age):
    print("Name:", name)
    print("Age:", age)

student_info(age=20, name="Ali")
```

A **default argument** is a parameter that has a predefined value. If the caller does not provide a value, Python uses the default value.

```
def greet(name="Student"):
    print("Hello", name)

greet()
greet("Ali")
```

The **return statement** is used to send a value from a function back to the part of the program where the function was called.

Python allows a function to return multiple values. These values are returned together and can be assigned to multiple variables.

```
def calculate(a, b):
    addition = a + b
    multiplication = a * b

    return addition, multiplication


sum_result, multiply_result = calculate(5, 4)

print("Addition:", sum_result)
print("Multiplication:", multiply_result)
```

Sometimes we do not know in advance how many positional arguments will be passed to a function. Python provides *args for this purpose.

The arguments are received as a **tuple**.

```
def calculate_sum(*numbers):

    total = 0

    for number in numbers:
        total += number

    return total


print(calculate_sum(10, 20))
print(calculate_sum(10, 20, 30, 40))
```

**kwargs allows a function to accept any number of keyword arguments. These arguments are stored as a dictionary.

```
def student_info(**details):

    for key, value in details.items():
        print(key, ":", value)


student_info(name="Ali", age=20, marks=85)
```

The global keyword is used when a function needs to modify a variable defined in the global scope.

```
number = 10


def change_number():

    global number

    number = 20


change_number()

print(number)
```

The **enumerate()** function is used when we need both the **index and the value** while iterating through a sequence. It eliminates the need to manually maintain a counter.

```
students = ["Ali", "Sara", "Ahmed"]

for index, student in enumerate(students):
    print(index, student)
```

ZIP() FUNCTION

The **zip()** function combines elements from two or more iterables based on their positions. It is particularly useful when working with related lists.

```
names = ["Ali", "Sara", "Ahmed"]
marks = [85, 90, 78]

for name, mark in zip(names, marks):
   print(name, mark)

names = ["Ali", "Sara", "Ahmed"]
marks = [85, 90, 78]

for number, (name, mark) in enumerate(
      zip(names, marks), start=1):

   print(number, name, mark)
```

LAMBDA FUNCTIONS

A **lambda function** is a small anonymous function that is defined without using the def keyword. Lambda functions are generally used when a simple operation is required for a short period of time.

A lambda function can accept any number of arguments but contains only a single expression.

```
square = lambda x: x * x

print(square(5))
add = lambda a, b: a + b

print(add(10, 20))
```

```
check_even = lambda x: "Even" if x % 2 == 0 else "Odd"

print(check_even(10))
print(check_even(7))
```

Lambda functions are frequently used with functions such as sorted() when we need to specify a custom sorting rule.

```
students = [
   ("Ali", 85),
   ("Sara", 92),
   ("Ahmed", 78)
```

```
]

students.sort(key=lambda student: student[1])

print(students)
```

LAB EXERCISES:

- Given a text:

```
text = """
machine learning is powerful
machine learning helps analyze data
data science uses machine learning
"""
```

Create a program that:

1. Converts text to lowercase.
2. Splits it into words.
3. Counts the frequency of each word using a dictionary.
4. Finds the most frequently occurring word.
5. Displays unique words using a set.
6. Displays words appearing more than once.

2. Two strings are called anagrams if they contain the same characters with the same frequencies, but the characters may appear in a different order.

Write a program to determine whether two given strings are anagrams.

- Create a function named is_anagram(s, t).
- Compare the frequency of each character in both strings.
- Return True if the strings are anagrams; otherwise return False.

3. Given a list of integers, determine whether any value appears more than once in the list.

Return True if at least one duplicate exists; otherwise return False.

1. Create a function named contains_duplicate(nums).
2. Use a set to keep track of values that have already been encountered.
3. If a value is already present in the set, identify it as a duplicate.

4. Given a list of integers, find the element that appears more than **n/2** times, where n is the total number of elements in the list.

You may assume that a majority element always exists.

1. Create a function named majority_element(nums).

2. Count the frequency of each element.
3. Store the frequencies using a dictionary.
4. Return the element whose frequency is greater than n/2.
5. You are given a list where each element represents the price of a stock on a particular day.

You may choose one day to buy the stock and a later day to sell it.

Your task is to determine the maximum possible profit.

If no profit can be made, return 0.

6. Create a function named max_profit(prices).
7. The buying day must occur before the selling day.
8. Return the maximum possible profit.

6. Given a list of integers and an integer k, find the k elements that occur most frequently in the list.

1. Create a function named top_k_frequent(nums, k).
2. Count the frequency of each element using a dictionary.
3. Rank the elements according to their frequency.
4. Return the k most frequent elements.
5. Use sorted() and a lambda expression for ranking.